Sistemas Operativos II

Práctica 2 — Informe 1

Carreras críticas en el productor-consumidor

Asignatura:	Sistemas Operativos II
Titulación:	Grado en Ingeniería Informática
Curso:	2024/25
Fecha:	10 de marzo de 2026

Universidad de Santiago de Compostela

Índice

1. Informe 1 — Carreras críticas en el productor-consumidor **2**
1.1. Descripción del problema 2
1.2. Cómo ejecutar los programas 2
1.3. Diseño y funcionamiento 2
1.4. Región crítica y cómo provocar carreras 3
1.5. Resultados observados y patrones de carrera 3
1.6. Conclusiones 4

1. Informe 1 — Carreras críticas en el productor-consumidor

1.1. Descripción del problema

El problema del productor-consumidor es un problema clásico de concurrencia en el que dos procesos comparten un buffer de tamaño fijo (`N = 10`). El productor escribe caracteres en el buffer y el consumidor los retira. Cuando no existe ningún mecanismo de sincronización, ambos procesos pueden acceder simultáneamente a las variables compartidas `count` y `top`, produciendo *carreras críticas*.

1.2. Cómo ejecutar los programas

Compilación

```
gcc -o prod prod.c
gcc -o cons cons.c
```

Ejecución (dos terminales distintas)

```
# Terminal 1:
./prod
# Terminal 2 (arrancar poco despues del productor):
./cons
```

El fichero de texto `input.txt` debe encontrarse en el mismo directorio de trabajo. El productor debe arrancarse primero para que cree el fichero de memoria compartida `/tmp/shm_buf`.

1.3. Diseño y funcionamiento

Memoria compartida

La memoria compartida se implementa mediante `mmap()` sobre el fichero `/tmp/shm_buf`. La estructura compartida contiene:

- `buf[N]`: buffer de tipo `char` que funciona como pila LIFO.
- `top`: índice de la cima de la pila (`top = 0` significa pila vacía).
- `count`: número de elementos actualmente en el buffer.

Funciones principales

- `produce_item()`: lee un carácter del fichero `input.txt` e incrementa el contador de la vocal correspondiente.
- `insert_item(item)`: inserta `item` en `buf[top]` y actualiza `top` y `count`. **Región crítica.**
- `remove_item()`: decrementa `top`, lee el carácter de la cima, deja un guión `'-'` en su lugar y decrementa `count`. **Región crítica.**
- `consume_item(item)`: contabiliza las vocales del carácter retirado.

Espera activa

Al no disponer de semáforos, la sincronización se realiza mediante *busy-waiting*: el productor comprueba `count == N` en un bucle y el consumidor comprueba `count == 0`, durmiendo un segundo en cada iteración de espera.

1.4. Región crítica y cómo provocar carreras

La región crítica comprende el acceso combinado a `count` y `top`. El código incluye líneas `sleep()` comentadas marcadas con `[SLEEP-RC]` que permiten ampliar artificialmente la duración de dicha región:

```c
void insert_item(char item)
{
    /* ... espera activa si buffer lleno ... */

    /* sleep(SLEEP_RC); */  /* [SLEEP-RC] amplifica la carrera */

    shm->buf[shm->top] = item;
    shm->top++;
    shm->count++;
}
```

Listing 1: Fragmento de `insert_item()` en `prod.c`

Cómo activar las carreras críticas

Modificar la constante `SLEEP_RC` en `prod.c` y `cons.c`. Al introducir un `sleep()` dentro de la región crítica, un proceso se congela antes de terminar de actualizar `top` y `count`, permitiendo al otro proceso entrar y operar sobre datos desfasados.

1.5. Resultados observados y patrones de carrera

Al modificar los valores de retardo dentro de la región crítica (`SLEEP_RC`), se observaron patrones de carrera muy específicos, fuertemente condicionados por las velocidades

relativas de los procesos, el límite del buffer (`N=10`) y el número total de iteraciones (`MAX_ITER=80`):

- **Productor lento (`sleep=1`) y Consumidor rápido (`sleep=0`):**
  Se observan carreras críticas de forma constante. El productor pasa un segundo entero "congelado" a mitad de su actualización de variables, lo que deja una ventana de vulnerabilidad enorme. El consumidor, al no tener retardo, entra repetidas veces a la región crítica y colisiona continuamente con la operación incompleta del productor.
- **Consumidor lento (`sleep=1`) y Productor rápido (`sleep=0`):**
  Las carreras se presentan de forma asimétrica. El productor **no** sufre carreras en sus primeras 10 iteraciones porque el buffer inicial está vacío y las inserta rápidamente antes de que el consumidor despierte de su primer letargo. Inversamente, el consumidor **no** sufre carreras en sus últimas 10 iteraciones porque el productor ya ha terminado sus 80 ciclos totales y abandonado la memoria compartida, dejando al consumidor operando en solitario.
- **Ambos rápidos (`sleep=0` en los dos):**
  Las carreras críticas ocurren casi exclusivamente en las iteraciones múltiples de 10. Al ejecutarse sin retardos artificiales, los procesos colisionan cuando alcanzan los límites físicos del buffer (0 o 10 elementos). Estos topes obligan a un proceso a entrar en *busy-waiting* frente a la región crítica. En el momento exacto en que el otro proceso altera el contador, ambos se abalanzan sobre la memoria compartida simultáneamente, garantizando la colisión exactamente cada 10 iteraciones.

El recuento final de vocales (cuando el código se lograba ejecutar sin corrupción fatal) fue idéntico en ambos procesos:

Vocal	Producidas	Consumidas
a / A	8	8
e / E	6	6
i / I	8	8
o / O	5	5
u / U	3	3

1.6. Conclusiones

- Sin sincronización, las carreras críticas son posibles siempre que la región crítica no sea atómica desde el punto de vista del planificador del sistema operativo.
- Los límites físicos del buffer actúan como cuellos de botella que fuerzan la sincronización temporal de los procesos, multiplicando la probabilidad de colisiones sistemáticas.
- El valor de `count` puede superar `N` o hacerse negativo si ambos procesos lo modifican concurrentemente.
- El índice `top` puede apuntar a una posición errónea, provocando sobreescritura o

lectura de datos corruptos en el buffer.

- La espera activa (*busy-waiting*) desperdicia CPU y no garantiza exclusión mutua: es una solución incorrecta e ineficiente.